

Universidade Federal do Paraná

Estrutura de dados I - Helcio padilha

André Dalberti da silva

Conteúdo para o Diário de Bordo

Semana 1: Estruturação e Implementação da Lista Estática (Eventos)

- **Responsabilidade Individual:** Fui responsável por toda a arquitetura inicial do sistema, englobando a criação do menu iterativo no main.c, a definição das estruturas de dados (struct) e a codificação das lógicas de cadastro, validação e busca de eventos (ev_cadastrar e ev_buscar).
- **Dificuldades Encontradas e Soluções:** A principal dificuldade foi decidir qual estrutura utilizar para armazenar os eventos sem adicionar complexidade desnecessária. Optei por uma lista linear estática (um vetor delimitado por MAX_EVENTOS em tempo de compilação), dado que a quantidade de eventos num setor de extensão é limitada e previsível, e o acesso direto pelo índice torna as buscas rápidas. O desafio técnico foi garantir que o limite do vetor não fosse ultrapassado (*overflow*). Resolvi isso bloqueando a inserção com a condição if (lista->tamanho >= MAX_EVENTOS) return 0; logo na primeira linha da função.

Trecho de Código Explicado:

```
22 int ev_cadastrar(ListaEventos *lista, int codigo, const char *nome, const char
   *data) {
23     if (lista->tamanho >= MAX_EVENTOS) return 0;
24     if (ev_buscar(lista, codigo) != -1) return -1; /* Duplicado */
25
26     int pos = lista->tamanho;
27     lista->dados[pos].codigo = codigo;
28     strncpy(lista->dados[pos].nome, nome, 49);
29     lista->dados[pos].nome[49] = '\0';
30     strncpy(lista->dados[pos].data, data, 10);
31     lista->dados[pos].data[10] = '\0';
32
33     lc_inicializar(&lista->dados[pos].participantes);
34
35     lista->tamanho++;
36     return 1;
37 }
```

Explicação: Esta função anexa um novo evento no final da lista estática. Num vetor, a

variável tamanho funciona duplamente: ela diz quantos itens existem e também serve como o índice exato da próxima vaga livre na memória. Utilizo esse índice para aceder diretamente à posição em $O(1)$, gravo os dados do evento, inicializo a lista encadeada vazia que guardará os participantes daquele evento, e avanço o tamanho lógico.

Semana 2: Lista Ligada com Nó Cabeçalho e Integração de Sistemas

- **Responsabilidade Individual:** Codifiquei toda a manipulação da lista ligada de participantes (`lc_inserir_fim`, `lc_remover_por_ra`, `lc_buscar_ra`), a gestão de memória e proteção contra vazamentos (`lc_destruir`) e as lógicas de negócio que cruzam eventos e alunos para gerar os relatórios de frequência.
- **Dificuldades Encontradas e Soluções:** Gerir as mudanças do ponteiro inicial da lista encadeada ao inserir o primeiro aluno ou remover o último, o que corria o risco de gerar falhas de segmentação (*Segmentation Fault*). A solução foi implementar a estrutura com um Nó Cabeçalho (Sentinela). Ao alocar um nó mestre que nunca é removido e não guarda dados do aluno, garanti que todo percurso e remoção sempre tenha um "nó anterior" disponível, erradicando os testes de `if (lista_vazia)` do meu código.

Trecho de Código Explicado:

```
54 int lc_remover_por_ra(ListaCabeçalho *lista, int ra) {  
55     No *anterior = lista->cabeçalho;  
56     while (anterior->proximo != NULL && anterior->proximo->ra != ra) {  
57         anterior = anterior->proximo;  
58     }  
59  
60     if (anterior->proximo == NULL) return 0; /* Nao encontrado */  
61  
62     No *remover = anterior->proximo;  
63     anterior->proximo = remover->proximo;  
64     free(remover);  
65     lista->tamanho--;  
66     return 1;  
67 }
```

Explicação: Para remover uma inscrição, preciso de estabilizar o percurso um passo atrás do alvo, para poder religar o encadeamento depois da exclusão. Início a busca diretamente no `lista->cabeçalho` (o sentinela) e navego até encontrar o RA. Ao identificar o alvo no próximo nó, ajusto o engate para "pular" o aluno cancelado, aplico o `free(remover)` para devolver a memória ao sistema de imediato e subtraio o contador.